Name: Shryitha Suddala

Hall Ticket:2303A51637

Batch:29

Assessment:11.2

Task 1: Stack Using AI Guidance

Prompt:

Write a python program to implement basic stack operations

Code:

```python
# Write a python program to implement basic stack operations
class Stack:
    def __init__(self):
        self.stack = []
    def push(self, item):
        self.stack.append(item)
    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        else:
            return "Stack is empty"
    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
        else:
            return "Stack is empty"
    def is_empty(self):
        return len(self.stack) == 0
    def size(self):
        return len(self.stack)
# Example usage
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    print("Top element:", stack.peek())  # Output: Top element: 3
    print("Stack size:", stack.size())   # Output: Stack size: 3
    print("Popped element:", stack.pop()) # Output: Popped element: 3
    print("Stack size after pop:", stack.size()) # Output: Stack size after pop: 2
    print("Is stack empty?", stack.is_empty()) # Output: Is stack empty? False
    stack.pop()
    stack.pop()
    print("Is stack empty after popping all elements?", stack.is_empty())
```

Output:

```
Top element: 3
Stack size: 3
Popped element: 3
Stack size after pop: 2
Is stack empty? False
Is stack empty after popping all elements? True
```

Significance:

A stack is an important data structure that follows the **Last In First Out (LIFO)** principle. It is widely used in applications such as **function calls, recursion, and undo/redo operations in editors**. Stacks also help in **expression evaluation and syntax parsing in compilers**.

Task 2: Queue Design

Prompt:

Create a Queue data structure following FIFO principles

Code:

```python
# Create a Queue data structure following FIFO principles
class Queue:
    def __init__(self):
        self.queue = []
    def enqueue(self, item):
        self.queue.append(item)
    def dequeue(self):
        if not self.is_empty():
            return self.queue.pop(0)
        else:
            return "Queue is empty"
    def front(self):
        if not self.is_empty():
            return self.queue[0]
        else:
            return "Queue is empty"
    def is_empty(self):
        return len(self.queue) == 0
    def size(self):
        return len(self.queue)
# Example usage
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)
    print("Front element:", queue.front())  # Output: Front element: 1
    print("Queue size:", queue.size()) # Output: Queue size: 3
    print("Dequeued element:", queue.dequeue()) # Output: Dequeued element: 1
    print("Queue size after dequeue:", queue.size()) # Output: Queue size after dequeue: 2
    print("Is queue empty?", queue.is_empty()) # Output: Is queue empty? False
    queue.dequeue()
    queue.dequeue()
    print("Is queue empty after dequeuing all elements?", queue.is_empty()) # Output: Is queue empty after dequeuing all elements? True
```

Output:

```
Front element: 1
Queue size: 3
Dequeued element: 1
Queue size after dequeue: 2
Is queue empty? False
Is queue empty after dequeuing all elements? True
```

Significance:

A queue follows the **First In First Out (FIFO)** principle, ensuring elements are processed in the order they arrive. It is commonly used in **CPU scheduling, printer task management, and network data handling**. Queues help manage tasks efficiently in systems where order is important.

Task 3: Singly Linked List Construction

Prompt:

Build a singly linked list supporting insertion and traversal.

Code:

```python
#build a singly linked list supporting insertion and traversal.
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        self.head = None
    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node
    def traverse(self):
        elements = []
        current = self.head
        while current:
            elements.append(current.data)
            current = current.next
        return elements
# Example usage
if __name__ == "__main__":
    linked_list = SinglyLinkedList()
    linked_list.insert(1)
    linked_list.insert(2)
    linked_list.insert(3)
    print("Elements in the linked list:", linked_list.traverse())  # Output: Elements in the linked list: [1, 2, 3]
```

Output:

```
Elements in the linked list: [1, 2, 3]
```

Significance:

A singly linked list is a dynamic data structure that allows **efficient insertion and deletion of elements**. It is useful when the **size of data changes frequently**. Linked lists are also used in implementing **stacks, queues, and memory management systems**.

Task 4: Binary Search Tree Operations

Prompt:

Implement a Binary Search Tree on insertion and traversal.

Code:

```python
class TreeNode:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
class BinarySearchTree:
    def __init__(self):
        self.root = None
    def insert(self, key):
        if self.root is None:
            self.root = TreeNode(key)
        else:
            self._insert_recursively(self.root, key)
    def _insert_recursively(self, node, key):
        if key < node.val:
            if node.left is None:
                node.left = TreeNode(key)
            else:
                self._insert_recursively(node.left, key)
        else:
            if node.right is None:
                node.right = TreeNode(key)
            else:
                self._insert_recursively(node.right, key)
    def inorder_traversal(self):
        return self._inorder_recursively(self.root)
    def _inorder_recursively(self, node):
        elements = []
        if node:
            elements.extend(self._inorder_recursively(node.left))
            elements.append(node.val)
            elements.extend(self._inorder_recursively(node.right))
        return elements
# Example usage
if __name__ == "__main__":
    bst = BinarySearchTree()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)
    bst.insert(6)
    bst.insert(8)
    print("Inorder traversal of the BST:", bst.inorder_traversal())  # Output: Inorder traversal of the BST: [2, 3, 4, 5, 6, 7, 8]
```

Output:

```
Inorder traversal of the BST: [2, 3, 4, 5, 6, 7, 8]
```

Significance:

A Binary Search Tree (BST) organizes data in a **sorted hierarchical structure**, allowing faster searching and insertion operations. It improves efficiency compared to linear search. BSTs are widely used in **database indexing, searching algorithms, and file systems**.

Task 5: Hash Table Implementation

Prompt:

Create a hash table with collision handling

Code:

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]
    def _hash(self, key):
        return hash(key) % self.size
    def insert(self, key, value):
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)  # Update existing key
                return
        self.table[index].append((key, value))  # Insert new key-value pair
    def get(self, key):
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None  # Key not found
    def delete(self, key):
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]  # Remove the key-value pair
                return True
        return False  # Key not found
# Example usage
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")
    print("Name:", hash_table.get("name"))  # Output: Name: Alice
    print("Age:", hash_table.get("age"))    # Output: Age: 30
    print("City:", hash_table.get("city"))  # Output: City: New York
    hash_table.delete("age")
    print("Age after deletion:", hash_table.get("age"))  # Output: Age after deletion: None
```

Output:

```
Name: Alice
Age: 30
City: New York
Age after deletion: None
```

Significance:

A hash table is used for **fast data storage and retrieval using key-value mapping**. It provides very efficient search operations with **average constant time complexity O(1)**. Hash tables are widely used in **databases, caches, dictionaries, and indexing systems**.